

```
from google.colab import files

uploaded = files.upload()
```

Choose Files 7 files

road81.png(image/png) - 191815 bytes, last modified: 7/7/2026 - 100% done
road82.png(image/png) - 568885 bytes, last modified: 7/7/2026 - 100% done
road83.png(image/png) - 582708 bytes, last modified: 7/7/2026 - 100% done
road84.png(image/png) - 562164 bytes, last modified: 7/7/2026 - 100% done
road85.png(image/png) - 504513 bytes, last modified: 7/7/2026 - 100% done
road86.png(image/png) - 538045 bytes, last modified: 7/7/2026 - 100% done
road87.png(image/png) - 485181 bytes, last modified: 7/7/2026 - 100% done

Saving road81.png to road81 (1).png
Saving road82.png to road82 (1).png
Saving road83.png to road83 (1).png
Saving road84.png to road84.png
Saving road85.png to road85.png
Saving road86.png to road86.png
Saving road87.png to road87.png

```
import os

print("Uploaded Files:")
print(os.listdir())
```

Uploaded Files:
['.config', 'road9.png', 'road80.png', 'road8.png', 'road84.png', 'road87.png', 'road83.png', 'road9 (1).png', 'road81 (1).png', 'road8

```
import cv2

img1 = cv2.imread("road81.png", 0)
img2 = cv2.imread("road82.png", 0)
img2 = cv2.imread("road83.png", 0)
img2 = cv2.imread("road84.png", 0)
if img1 is None:
    print("Error: Image 'road88.png' not found. Please upload it to
if img2 is None:
    print("Error: Image 'road79.png' not found. Please upload it to

if img1 is not None and img2 is not None:
    print("Images Loaded Successfully")
else:
    print("Failed to load one or both images.")
```

Images Loaded Successfully

```
fast = cv2.FastFeatureDetector_create()

kp_fast1 = fast.detect(img1, None)
kp_fast2 = fast.detect(img2, None)

print("FAST Corners in Image 1:", len(kp_fast1))
print("FAST Corners in Image 2:", len(kp_fast2))
```

FAST Corners in Image 1: 326
FAST Corners in Image 2: 2613

```
orb = cv2.ORB_create(nfeatures=1000)

kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

print("ORB Keypoints in Image 1:", len(kp1))
print("ORB Keypoints in Image 2:", len(kp2))
```

ORB Keypoints in Image 1: 664
ORB Keypoints in Image 2: 878

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

matches = bf.match(des1, des2)

matches = sorted(matches, key=lambda x: x.distance)

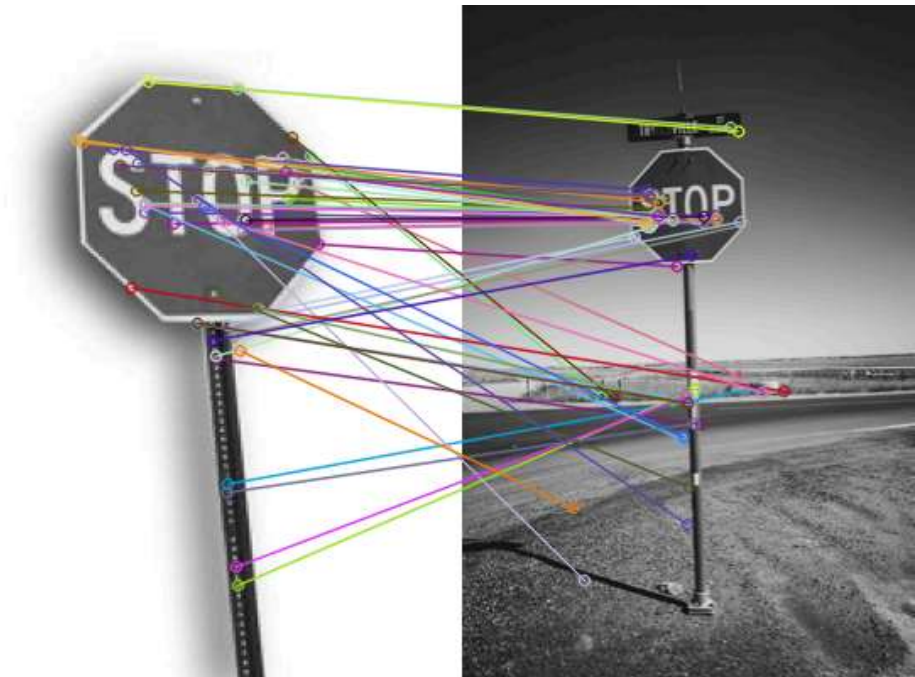
print("Total Matches:", len(matches))
```

Total Matches: 191

```
from google.colab.patches import cv2_imshow

result = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
    matches[:50],
    None,
    flags=2
)

cv2_imshow(result)
```



```
if len(matches) > 50:
    print("Road Sign Matched")
else:
    print("Road Sign Not Matched")
```

Road Sign Matched

```
import cv2
from google.colab.patches import cv2_imshow

# Load images
img1 = cv2.imread("road81.png", 0)    # Reference
img2 = cv2.imread("road82.png", 0)    # Test
img2 = cv2.imread("road83.png", 0)

# FAST Corner Detection
fast = cv2.FastFeatureDetector_create()

fast_kp1 = fast.detect(img1, None)
fast_kp2 = fast.detect(img2, None)

print("FAST Corners in Reference Image :", len(fast_kp1))
print("FAST Corners in Test Image      :", len(fast_kp2))
```

```
# ORB Feature Extraction
orb = cv2.ORB_create(nfeatures=1000)

kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

print("\nORB Keypoints in Reference Image :", len(kp1))
print("ORB Keypoints in Test Image      :", len(kp2))

# Feature Matching
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda x: x.distance)

print("\nTotal Matches Found :", len(matches))

# Matching Percentage
max_possible = min(len(kp1), len(kp2))
match_percentage = (len(matches) / max_possible) * 100

print("Matching Percentage = {:.2f}%".format(match_percentage))

# Display First 10 Matched Coordinates
print("\nFirst 10 Matched Coordinates")

for i, m in enumerate(matches[:10]):
    pt1 = kp1[m.queryIdx].pt
    pt2 = kp2[m.trainIdx].pt

    print(f"\nMatch {i+1}")
    print("Reference Image Coordinate :", pt1)
    print("Test Image Coordinate      :", pt2)
    print("Distance                      :", round(m.distance,2))

# Draw Matches
result = cv2.drawMatches(
    img1,
    kp1,
    img2,
    kp2,
    matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

cv2_imshow(result)

# Recognition Decision
print("\nRecognition Result")

if match_percentage > 60:
    print("Very Strong Match")
elif match_percentage > 40:
    print("Good Match")
elif match_percentage > 20:
```

```
print("Partial Match")
else:
    print("Weak Match")
```

FAST Corners in Reference Image : 326
FAST Corners in Test Image : 4667

ORB Keypoints in Reference Image : 664
ORB Keypoints in Test Image : 948

Total Matches Found : 170
Matching Percentage = 25.60%

First 10 Matched Coordinates

Match 1
Reference Image Coordinate : (153.0, 204.0)
Test Image Coordinate : (188.40000915527344, 84.0)
Distance : 29.0

Match 2
Reference Image Coordinate : (153.60000610351562, 207.60000610351562)
Test Image Coordinate : (110.48143768310547, 235.89279174804688)
Distance : 39.0

Match 3
Reference Image Coordinate : (139.0, 92.0)
Test Image Coordinate : (188.6400146484375, 83.52000427246094)
Distance : 43.0

Match 4
Reference Image Coordinate : (160.8000030517578, 288.0)
Test Image Coordinate : (187.20001220703125, 77.76000213623047)
Distance : 44.0

Match 5
Reference Image Coordinate : (164.40000915527344, 321.6000061035156)
Test Image Coordinate : (128.3973388671875, 235.89279174804688)
Distance : 46.0

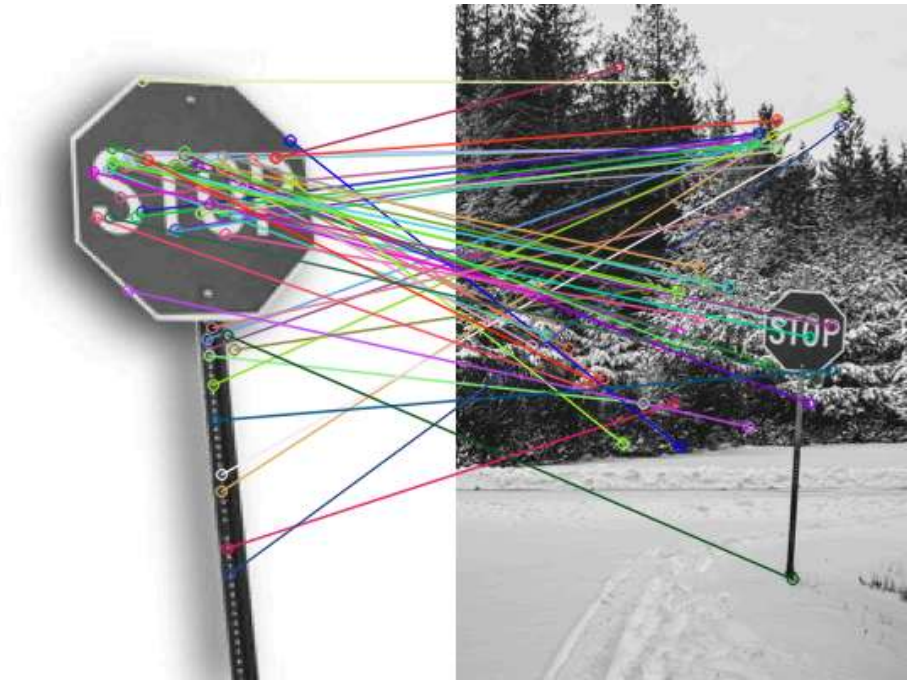
Match 6
Reference Image Coordinate : (106.0, 89.0)
Test Image Coordinate : (169.20001220703125, 85.20000457763672)
Distance : 47.0

Match 7
Reference Image Coordinate : (168.4095458984375, 204.24136352539062)
Test Image Coordinate : (111.97441864013672, 167.9616241455078)
Distance : 47.0

Match 8
Reference Image Coordinate : (101.0, 114.0)
Test Image Coordinate : (174.24000549316406, 79.20000457763672)
Distance : 49.0

Match 9
Reference Image Coordinate : (133.20001220703125, 133.20001220703125)
Test Image Coordinate : (171.36000061035156, 79.20000457763672)
Distance : 49.0

Match 10
Reference Image Coordinate : (161.24317932128906, 121.82818603515625)
Test Image Coordinate : (208.8000030517578, 196.8000030517578)
Distance : 49.0



Recognition Result
Partial Match

```
import matplotlib.pyplot as plt

signs = ["road100", "road101", "road102", "road103"]
percentages = [82, 65, 35, 90] # Example values

plt.figure(figsize=(8,5))
```

```
plt.bar(signs, percentages)

plt.title("Road Sign Matching Comparison")
plt.xlabel("Reference Signs")
plt.ylabel("Matching Percentage (%)")

for i, v in enumerate(percentages):
    plt.text(i, v + 1, f"{v:.1f}%")

plt.show()
```

